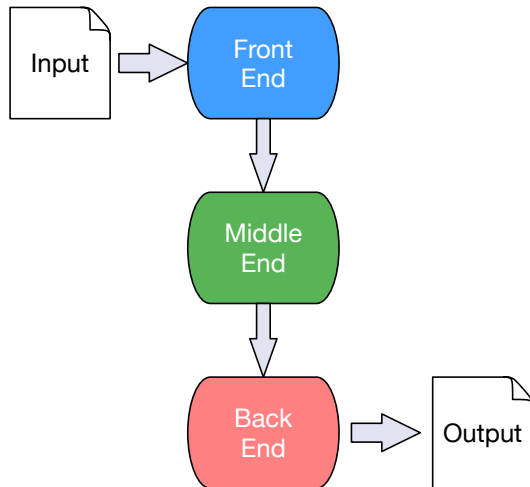


Compiler Construction

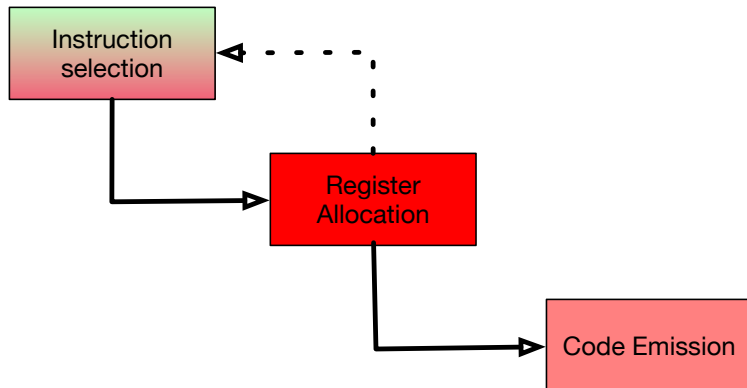
⋈ Backend ⋈

General layout – Compiler structure



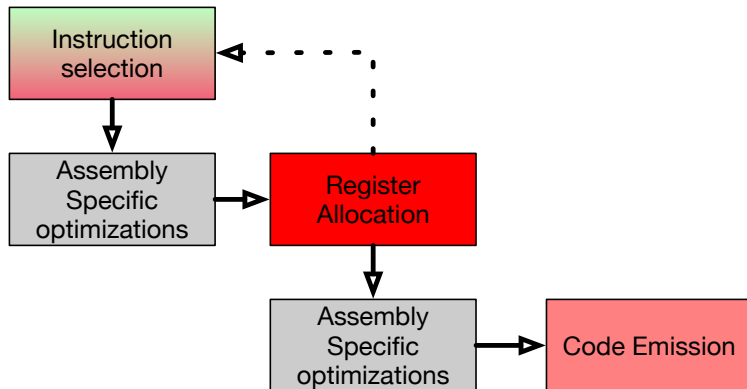
Backend's overview (1/2)

Everything hardware specific should go in the backend



Backend's overview (2/2)

Everything hardware specific should go in the backend



Pre Register Allocation

- Control Flow Graph
- Liveness Analysis
- Dataflow Analysis
- Static Single Assignment
- Interference graph
- Loop unrolling
- Cache handling

Compute information necessary to register allocation

Register Allocation

- Graph Coloring
- Spilling
- Coalescing

Find a register for each variable
OR
put some variables onto the stack

Post Register Allocation

- Instruction scheduling
- Code emission
- *Garbage collection*

Assembly / Assembler / Machine Code

Assembly Code Program

Human-readable text file

Machine Code

Binary code stored in memory, directly executable by hardware

Assembler

Program that translates assembly into machine code

Each architecture has its own machine code and assembly:

x86, ARM, RISC-V, AVR, 8051, ...

Problem statement

Quick Overview of microprocessors

Choose one microprocessor for this course

- ⇒ Easy to understand
- ⇒ Fit modern architectures
- ⇒ Correspond to most of the actual compilers

Instruction set

“ *Instruction set architecture is the structure of a computer that a machine language programmer (or a compiler) must understand to write a correct (timing independent) program for that machine*

IBM introducing 360 (1964)

The Instruction Set Architecture (ISA) is the part of the processor that is visible to the programmer or compiler writer.

Architecture vs Microprocessor

Instruction Set Architecture (ISA) - Abstract machine model

- Visible to programmer / compiler
- Defines instructions, registers, memory model
- Example: RISC-V, x86, ARM

Microprocessor - Physical hardware chip

- Implements one ISA
- Defines pipeline, cache, clock rate, execution units
- Example: Intel Core i7, Apple M2

For this course

The compiler targets an **ISA**, not a specific microprocessor.

What is an ISA? (Instruction Set Architecture)

An instruction set specifies a processor functionality:

- what operations are supported
- what storage mechanisms are used
- how to access storage
- how to communicate with other devices in the system

What is an ABI? (Application Binary Interface)

Definition

The ABI defines how compiled code interacts at the binary level.

It specifies:

- How function arguments are passed (registers vs stack)
- Where return values are stored
- Which registers must be preserved
- Stack layout and alignment
- Data type sizes and memory layout
- System call conventions

Compiler perspective

Your compiler must follow the ABI, otherwise it cannot link with libraries or OS code.

Technical aspect of instruction set

- ❶ format: length, encoding
- ❷ operations: data type (floating or fixed point), number and kind of operands
- ❸ storage:
 - ▶ internal: accumulator, stack, register
 - ▶ memory: address size, word size, addressing modes
- ❹ control: branch condition, support for procedures, predication

What makes a good instruction set?

An instruction set specifies a processor functionality:

- **implementability**: support for a (high performance) range of implementation
- **programmability**: easy to express program (by Humans before 80's, mostly by compiler nowadays)
- **backward/forward compatibility**: implementability & programmability across generations

cisc – Complex Instruction Set Chip

- large number of instructions (100-250)
- usually relatively small amount of registers, with some having special purposes
- arithmetic operations can be performed directly in memory
- two address code
- many possible effects (e.g., self-incrementation)

CISC – Pros & Cons

Pros:

- Simplified compiler
- Small assembly code
- Few instructions will be fetched
- Special purpose register available: stack pointer, interrupt handling ...

Cons:

- Variable length instruction format
- Variable number of clock cycles per instruction
- Limited number of general purpose registers

Motivations for something else!

CISC assembly code may be shorter than RISC, but this is not necessarily the case after it is assembled into machine code.

The complex instructions can make optimizing harder: even decoding an instruction can take several clock cycles.

RISC architectures, on the other hand, were designed with the goal of executing one instruction per clock cycle.

RISC – Reduced Instruction Set Chip

- usually more generic purpose registers
- arithmetic only available on registers
- 3 address code
- load and store relative to a register ($M[r + \text{const}]$)
- only one effect or result per instruction

RISC – Pipeline 1/3

Pipelining is the overlapping the execution of several instructions in a pipeline fashion.

A RISC pipeline is (typically) decomposed into five stages:

- 1 Instruction Fetch (IF)
- 2 Instruction Decode (ID)
- 3 Execute (EX)
- 4 Memory Access (MA)
- 5 Write Back (WB)

RISC – Pipeline 2/3

inst1:	IF	ID	EX	MA	WB			
inst2:		IF	ID	EX	MA	WB		
inst3:			IF	ID	EX	MA	WB	
inst4:				IF	ID	EX	MA	WB
inst5:					IF	ID	EX	MA WB

The slowest stage determines the speed of the whole pipeline!

Ex introduces latency

- Register-Register Operation: 1 cycle
- Memory Reference: 2 cycles
- Multi-cycle Instructions (floating point): many cycles

RISC – Pipeline 3/3

Data hazard: When an instruction depends on the results of a previous instruction still in the pipeline.

- inst1 write in \$s1 during WB
- inst1 read in \$s1 during ID

inst1:	IF	ID	EX	MA	WB	
inst2:		IF	ID	EX	MA	WB

inst2 must be split, causing delays...

other dependencies can appear

RISC – Pros & Cons

Pros:

- Fixed length instructions: decoding is easier (and consistently fast)
- Simpler hardware: higher clock rate
- Efficient Instruction pipeline
- Large number of general purpose registers
- Overlapped register windows to speed up procedure call and return
- One instruction per cycle

Cons:

- Minimal number of addressing modes: only Load and Store
- Relatively few different instructions

Nowadays

- the classification pure-RISC or pure-CISC is becoming more and more inappropriate and may be irrelevant
 - modern processors use a combination of elements from both design styles
 - Some processors are classified as CISC while employing a number of RISC features, such as pipelining
-
- ARM provides some advantages of using a CISC (in terms of functionality) and some advantages of an RISC (in terms of reduced code lengths).
 - Modern x86 CPUs have a very advanced decoder that reads the CISC instructions and... translates them to a RISC microarchitecture instructions.

Lessons to be learned

Implementability

Driven by technology: microcode, VLSI, FPGA, pipelining, superscalar, SIMD, SSE

Programmability

Driven by compiler technology

Sum-up

- Many non technical issues influence ISA's
- Best solutions don't always win (Intel X86)

Intel X86 (IA32)

- Introduced in 1978
- 8×32 bits "general" register
- variable length instructions (1–15 byte)

Intel's trick?

Decoder translates CISC into RISC micro-operations

A modern open RISC instruction set architecture

- Designed at UC Berkeley
- Open standard ISA
- Modular extensions (M, F, D, C, ...)
- Used in embedded systems, academia, industry

In this course

We focus on **RV64**, but everything applies to **RV32** as well.

RISC-V Registers (RV32I and RV64I)

Name	ABI	Usage
x0	zero	Constant 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5–x7	t0–t2	Temporaries
x8	s0/fp	Saved register / Frame pointer
x9	s1	Saved register
x10–x17	a0–a7	Arguments / return values
x18–x27	s2–s11	Saved registers
x28–x31	t3–t6	Temporaries

- 32 general-purpose registers
- x0 is always 0
- Caller-saved: t^* , a^*
- Callee-saved: s^*

RISC-V instructions are always **32 bits**.

- The CPU always fetches 4 bytes
- The next instruction is always at $PC + 4$
- No need to determine instruction length first

Inside those 32 bits you must encode:

- opcode
- destination register
- source register(s)
- function bits
- immediate (if any)

Arithmetic Instructions

- `add rd, rs1, rs2`
- `addi rd, rs1, imm`
- `sub rd, rs1, rs2`
- `lui rd, imm` (Load upper immediate)
- `auipc rd, imm`

Arithmetic width:

- RV32: 32-bit operations
- RV64: 64-bit operations
- Wrap-around on overflow
- No overflow exceptions

Exercise - 1

Translate to RISC-V:

```
move temp t0 const 42  
move temp t1 const 10  
move temp t2 binop add temp t1 temp t2
```

Exercise - 1

Translate to RISC-V:

```
move temp t0 const 42
move temp t1 const 10
move temp t2 binop add temp t1 temp t2
```

```
addi x5, x0, 42    # t0 = 42
addi x6, x0, 10    # t1 = 10
add x7, x5, x6     # t2 = t0 + t1
```


lui - Load Upper Immediate

```
lui rd, imm20
```

Effect

$$rd = imm20 \ll 12$$

lui - Load Upper Immediate

```
lui rd, imm20
```

Effect

$$rd = imm20 \ll 12$$

Bit layout (RV32)

31	12	11	0
imm20		00000000000000	

lui - Load Upper Immediate

```
lui rd, imm20
```

Effect

$$rd = imm20 \ll 12$$

Bit layout (RV32)

31	12	11	0
imm20		00000000000000	

Example

```
lui t0, 0x12345 # t0 holds 0x12345000
```

lui - Load Upper Immediate

```
lui rd, imm20
```

Effect

$$rd = imm20 \ll 12$$

Bit layout (RV32)

31	12	11	0
imm20		000000000000	

Example

```
lui t0, 0x12345 # t0 holds 0x12345000
```

Used to build 32-bit constants

```
lui t0, 0x12345  
addi t0, t0, 0x678 # t0 = 0x12345678
```

Load Immediate (li)

Pseudo-instruction

```
li t0, 5
```

Load Immediate (li)

Pseudo-instruction

```
li t0, 5
```

Assembler expands to:

```
addi t0, x0, 5
```

Load Immediate (li)

Pseudo-instruction

```
li t0, 5
```

Assembler expands to:

```
addi t0, x0, 5
```

Large constants

```
li t0, 0x12345678
```

Load Immediate (li)

Pseudo-instruction

```
li t0, 5
```

Assembler expands to:

```
addi t0, x0, 5
```

Large constants

```
li t0, 0x12345678
```

Assembler expands to:

```
lui  t0, 0x12345  
addi t0, t0, 0x678
```


Multiplication and Division (M Extension)

- `mul rd, rs1, rs2`
- `mulh rd, rs1, rs2`
- `div rd, rs1, rs2`
- `divu rd, rs1, rs2`
- `rem rd, rs1, rs2`
- `remu rd, rs1, rs2`

Unlike MIPS:

- No special hi/lo registers
- Results written directly to `rd`

Logical Instructions

- `and`, `andi`
- `or`, `ori`
- `xor`, `xori`
- `sll`, `slli`
- `srl`, `srli`
- `sra`, `srai`

Comparison:

- `slt`, `slti`
- `sltu`, `sltiu`

Branches

- `beq rs1, rs2, label`
 - `bne rs1, rs2, label`
 - `blt, bge`
 - `bltu, bgeu`
 - `j label`
-
- PC-relative addressing
 - No condition flags
 - Comparison is built into branch instruction

Exercise - 2

Translate to RISC-V:

```
cjump lt temp t0 temp t1 Ltrue Lfalse
label Lfalse
move temp t3 const 1
jump Lend
label Ltrue
move temp t3 const 0
label Lend
```

Exercise - 2

Translate to RISC-V:

```
cjump lt temp t0 temp t1 Ltrue Lfalse
label Lfalse
move temp t3 const 1
jump Lend
label Ltrue
move temp t3 const 0
label Lend
```

```
blt x5, x6, Ltrue
Lfalse:
li x7, 1
j Lend
Ltrue:
li x7, 0
Lend:
```

Exercise - 3

Translate to RISC-V:

```
label Lstart  
cjump lt temp t0 const 10 Lend Lbody  
label Lbody  
add temp t0 temp t0 const 1  
jump Lstart  
label Lend
```

Exercise - 3

Translate to RISC-V:

```
label Lstart
cjump lt temp t0 const 10 Lend Lbody
label Lbody
add temp t0 temp t0 const 1
jump Lstart
label Lend
```

```
Lstart:
li x6, 10
blt x5, x6, Lend
Lbody:
addi x5, x5, 1
j Lstart
Lend:
```

Jumps & Link

- `jal rd, label`
- `jalr rd, rs1, imm`

Typical function call:

- `jal ra, function`
- return via `jalr x0, ra, 0`

Exercise - 4

Translate to RISC-V:

```
seq
  sxp call name print_int temp t0
  move temp t2 call name float_of_int temp t1 call end
seq end
```

Exercise - 4

Translate to RISC-V:

```
seq
  sxp call name print_int temp t0
  move temp t2 call name float_of_int temp t1 call end
seq end
```

```
mv a0, x5      # a0 is first argument
jal print_int  # call
mv a0, x6
jal float_of_int # call
mv x7, a0      # a0 is the return value
```

Load Instructions

- lb, lbu
- lh, lhu
- lw (32-bit)
- ld (64-bit, RV64)

Syntax:

```
lw rd, offset(rs1)
```

Store Instructions

- sb
- sh
- sw (32-bit)
- sd (64-bit, RV64)

Example:

```
sw rs2, offset(rs1)
```

Floating Point

- 32 FP registers: `–f0f31`
- Single precision (F)
- Double precision (D extension)
- Dedicated Instructions

Floating Point Arithmetic - Single precision

- `fadd.s`
- `fsub.s`
- `fmul.s`
- `fdiv.s`
- `fsqrt.s`

Comparison:

- `feq.s`
- `flt.s`
- `fle.s`

Floating Point Arithmetic - Double precision

- `fadd.d`
- `fsub.d`
- `fmul.d`
- `fdiv.d`
- `fsqrt.d`

Comparison:

- `feq.d`
- `flt.d`
- `fle.d`

RV32 vs RV64

Same philosophy

- Same instruction formats
- Same registers (x0–x31)
- Same ABI convention
- Same extensions (M, F, D, ...)

Main difference

- Register width (32 vs 64 bits)
- Address space size
- Additional RV64 instructions (e.g., addw, ld, sd)

Compiler perspective

Register allocation and instruction selection are identical. Only data width changes.

RV64 Registers and Integer Types

Fixed size registers

In RV64, all general-purpose registers are 64-bit wide.

C source:

```
int  x;    // 32-bit  
long y;    // 64-bit
```

Both variables live in 64-bit registers:

```
# x stored in a0  (64-bit register)  
# y stored in a1  (64-bit register)
```

Important

There is no separate 32-bit register file.

32-bit vs 64-bit Arithmetic in RV64

C source:

```
int  x = a + b;  
long y = c + d;
```

Generated code:

```
# 32-bit addition (int)
```

```
addw  a0, a1, a2
```

```
# 64-bit addition (long)
```

```
add   a3, a4, a5
```

Difference

- add → full 64-bit operation
- addw → operates on lower 32 bits then sign-extends to 64 bits

addw - Why?

Example:

```
int x = -1;
```

Suppose register contains:

```
a0 = 0x00000000FFFFFFFF
```

After:

```
addw a0, a0, zero
```

Register becomes:

```
a0 = 0xFFFFFFFFFFFFFFFF
```

Compiler rule

32-bit results in RV64 must be properly sign-extended to 64 bits.

addw - How?

We saw:

`a0 = 0x00000000FFFFFFFF`

Split into 32-bit halves:

Upper 32 bits : 00000000

Lower 32 bits : FFFFFFFF

In 32-bit two's complement:

`0xFFFFFFFF` = -1

But in 64-bit two's complement:

-1 = `0xFFFFFFFFFFFFFFFF`

Why addw is needed

addw sign-extends the lower 32 bits to produce the correct 64-bit value.

Integer vs Floating Constants

C source:

```
int    x = 42;  
float  y = 3.14f;
```

Integer constant:

```
li a0, 42
```

Floating constant:

cannot encode 3.14f directly

Difference

Integer immediates are encoded in instructions.

Floating-point constants are stored in memory.

Floating Constants Live in Data Sections

Typical assembly output:

```
.section .rodata  
.LC0:  
    .float 3.14
```

Then loaded at runtime:

```
la    t0, .LC0  
flw   f0, 0(t0)
```

Why .rodata?

Floating constants are read-only and shared across the program.

Why Floats Cannot Be Immediate

RISC-V instructions are 32 bits long.

Single precision float = 32 bits.

There is no room for:

```
fadd.s f0, f1, 3.14f    # not possible
```

Instead:

```
# load constant first  
flw f2, .LC0(t0)  
fadd.s f0, f1, f2
```

Compiler implication

Float constants require memory allocation and symbol management.

Large Integers Also Use Data

Small integer:

```
li a0, 42
```

Large 64-bit integer:

```
long x = 0x1234567812345678;
```

Generated code:

```
lui    a0, 0x12345  
addi   a0, a0, 0x678  
...
```

Important

Integers may use immediates or instruction sequences.

Floating constants always require memory.

Summary

- Clean ISA
- No special registers (like MIPS hi/lo)
- No condition flags
- Modular architecture
- Well-suited for teaching compilers